

# Base Github workflows for Ansible

Тимофей Семисоха

## Base Github workflows for Ansible

В этом небольшом тексте рассмотрим как можно использовать github workflows для автоматического тестирования ansible плейбуков в репозитории с помощью molecule

### Структура репозитория

Для работы workflows нужно, чтобы репозиторий был построен по следующему принципу:

```
.github/workflows
├── molecule.yml
└── molecule
    ├── ci
    │   ├── converge.yml
    │   ├── molecule.yml
    │   └── verify.yml
    └── default
        ├── converge.yml
        ├── molecule.yml
        └── verify.yml
```

Суть следующая - у нас есть основные папки, это .github для настроек, о которых мы сообщаем github. И molecule, в которой соблюдается классическая схема организации папок для Molecule.

### Папки для Molecule

Внутри указаны базовые группы сценариев, это ci и default. Можно сказать, что это сами тесты, которые мы хотим проводить. Если бы мы хотели создать еще один тест, то в той же папке molecule нужно было бы создать папку "onemoretest".

В каждом из таких тестов лежат сценарии, по которым мы собираемся проводить тестирование. В структуре указаны минимальные три файла, с которыми Molecule может отработать. Если нужно провести более серьезное тестирование, в папку к имеющимся сценариям можно добавить остальные, приемлемые для Molecule (например prepare.yml).

На десктопе при такой структуре файлов default тест запускался бы командой molecule test. Тест ci запускался бы командой molecule test -s ci

## Workflows

Перейдем к папке `.github` с действиями для гитхаба. Для тестов с помощью молекулы нам понадобится файл `molecule.yml` (в репозитории `molecule_old.yml`), в котором мы расскажем гитхабу, каких действий от него ожидаем.

```
1 name: Molecule Test
2
3 on:
4   push:
5     branches: [ main, master ]
6   pull_request:
7     branches: [ main, master ]
8
9 jobs:
10  molecule:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout code
15        uses: actions/checkout@v4
16
17      - name: Set up Python
18        uses: actions/setup-python@v5
19        with:
20          python-version: "3.12"
21
22      - name: Install dependencies
23        run: |
24          python -m pip install --upgrade pip
25          pip install molecule molecule-docker ansible-lint
26          pip install --upgrade molecule-docker
27
28
29      - name: Run Molecule tests (ci scenario)
30        run: molecule test -s default
31    env:
32      PY_COLORS: "1"
33      ANSIBLE_FORCE_COLOR: "1"
34      ANSIBLE_ALLOW_BROKEN_CONDITIONALS: "true"
```

После добавления такого файла в указанную директорию, Github начнет тестировать наш репозиторий с помощью Molecule после каждого изменения в основной ветке. Рассмотрим подробнее, из чего состоит этот файл, и чем конкретно будет заниматься Github.

```

1 name: Molecule Test
2
3 on:
4   push:
5     branches: [ main, master ]
6   pull_request:
7     branches: [ main, master ]

```

Это верхушка нашего файла. Здесь указано название сценария, под ним мы указываем после каких действий мы хотим проводить тестирование. В примере указаны пуши и пул реквесты в мейн.

```

1 jobs:
2   molecule:
3     runs-on: ubuntu-latest

```

Здесь указываем работы, которые нужно будет провести, и с помощью каких инструментов. В нашем случае говорим, что хотим контейнер с ubuntu-latest, в котором будет запущена Molecule. (Контейнеры, которые создает Molecule будут созданы внутри этого контейнера)

```

1   steps:
2     - name: Checkout code
3       uses: actions/checkout@v4

```

Здесь и далее перечисляются шаги, которые будут выполнены для тестирования. actions / checkout@v4 используется для загрузки файлов из репозитория в контейнер в котором будет запускаться Molecule.

```

1 - name: Set up Python
2   uses: actions/setup-python@v5
3   with:
4     python-version: "3.12"

```

Здесь Github скачивает python версии "3.12" и добавляет его в PATH. Все последующие команды будут выполняться этим python, а не встроенным в Ubuntu.

```

1   - name: Install dependencies
2     run: |
3       python -m pip install --upgrade pip
4       pip install molecule molecule-docker ansible-lint
5       pip install --upgrade molecule-docker

```

Скачиваем Molecule, драйвер для Docker и утилиту для статического анализа скриптов ansible

```

1     - name: Run Molecule tests (ci scenario)
2       run: molecule test -s default
3       env:
4         PY_COLORS: "1"
5         ANSIBLE_FORCE_COLOR: "1"
6         ANSIBLE_ALLOW_BROKEN_CONDITIONALS: "true"

```

Запускаем тест default. Ниже устанавливаются цвета для логов. Дополнительно здесь указан параметр для замены ошибок, связанных с некорректными условиями, на предупреждения. По почти подтвержденной информации, в старых версиях скриптов можно было полагаться на истинность небулевых типов, а сейчас так нельзя. Для совместимости со старой логикой можно воспользоваться ANSIBLE BROKEN CONDITIONALS.

Возможно, это было важно именно для моего плейбука. Но если столкнетесь с ошибками логики, отдаленно связанными с вашим ansible скриптом, можете попробовать добавить эту строку.

## Upgrade

Можно произвести апгрейд системы в нашем репозитории. Реализуем следующие пожелания:

- Тестировать плейбуки с произвольным расположением в репозитории
- Передавать в тестируемый плейбук различные переменные окружения

Первая задача решается следующим образом: можем завести в репозитории папку playbooks и класть туда все, что хотим протестировать (converge в этом случае можно удалить). А в файле molecule.yml в папке теста (/molecule/ci) нужно указать явно, из какой директории берем тестируемый файл (см в репозитории):

```

1     playbooks:
2     converge: ../../playbooks/jupyterlab.yml

```

Для передачи переменных окружения мы должны указать в тестируемом файле vars. Например для всего play:

```

1     ---
2     - name: Install jupyter lab
3       hosts: all
4       become: yes
5       vars:
6         jupyter_port: "{{ lookup('env', 'JUPYTER_PORT') | default
7           ('14500', true) }}"
6         jupyter_user: "{{ lookup('env', 'JUPYTER_USER') | default('
          jupusr', true) }}"

```

А значения этих переменных указать в workflow в env:

```
1      - name: Run Molecule tests (ci scenario)
2      run: molecule test -s default
3      env:
4      PY_COLORS: "1"
5      ANSIBLE_FORCE_COLOR: "1"
6      ANSIBLE_ALLOW_BROKEN_CONDITIONALS: "true"
7      JUPYTER_PORT: "14500"
8      JUPYTER_USER: "jupusr"
```